

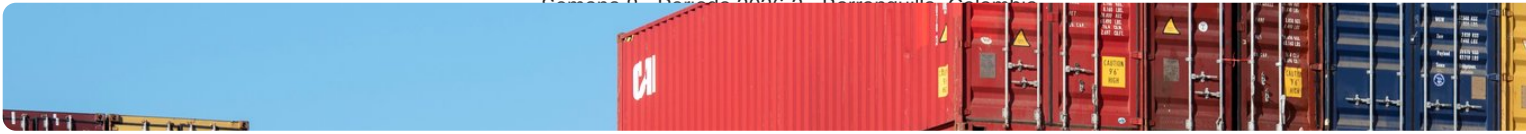
Contenedores y registro de imágenes

Unidad 2 · Integración de Soluciones para Plataformas Cloud

Ing. Rodolfo Cañas Cervantes

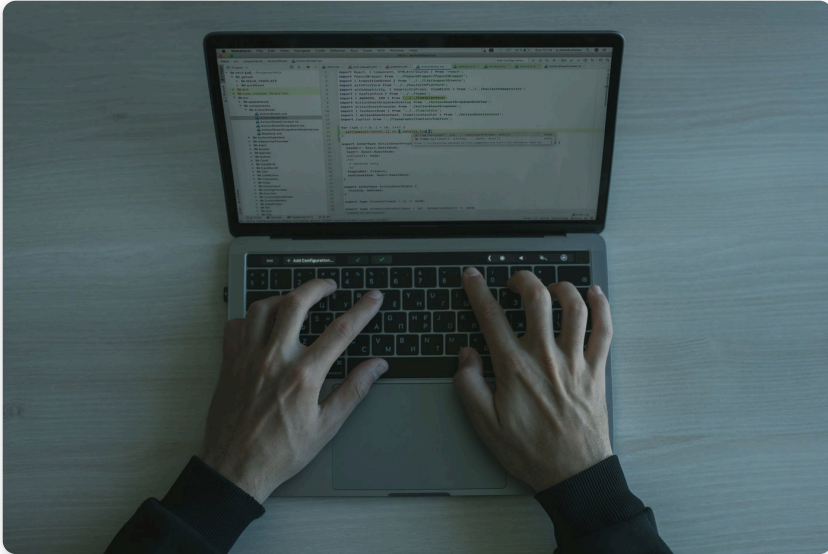
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 8 · Período 2026-2 · Barranquilla, Colombia



En mi máquina sí funciona

El entorno de cada desarrollador difiere del de producción: la misma app se comporta distinto en cada lugar.



El síntoma

La aplicación corre en el portátil del desarrollador y **falla al desplegarse**.

Las causas

- Versiones de runtime distintas
- Librerías instaladas a mano
- Configuración implícita del entorno

La promesa del contenedor

Empaquetar la app **con su entorno**, una vez, para cualquier lugar.

Sin virtualizar, el entorno de la app es el propio servidor: se configura a mano y se hereda entre despliegues.



El entorno es el servidor

La app depende de la configuración instalada a mano en esa máquina.

Capacidad desperdiciada

Hardware dedicado al 100 % de una sola aplicación, incluso fuera de pico.

Despliegue manual

Cada servidor se configura uno a uno — **nunca idénticos**.

La virtualización parte el hardware

El hipervisor convierte un servidor en varias máquinas virtuales — cada una con su sistema operativo completo.



El hipervisor

Particiona un servidor en máquinas virtuales independientes.

Una VM = un SO completo

GB de disco y **minutos** de arranque por máquina.

El salto de eficiencia

Varias apps por servidor — pero cada una sigue con su SO entero.

VM vs. contenedor: qué comparte cada uno

La VM virtualiza el hardware; el contenedor virtualiza el sistema operativo — menos peso, arranque en segundos.



Máquina virtual

- Sistema operativo completo por VM — de GB
- Arranque en minutos
- Aislamiento vía hardware virtual
- Una VM por app o por grupo



Contenedor

- Comparte el kernel del host — de MB
- Arranque en segundos
- Aislamiento vía namespaces + cgroups
- Muchos contenedores por host

Namespaces y cgroups: las dos piezas del kernel

Namespaces: qué ve el contenedor. cgroups: cuánto consume del host. Nada más hace falta.



Namespaces — qué ve

- Su propia red, sus propios procesos
- Su propio sistema de archivos de montaje
- Cada contenedor cree que está solo en la máquina



cgroups — cuánto consume

- Límites de CPU y de memoria por contenedor
- Un contenedor hambriento no se come al vecino
- Medible y aplicable en caliente

Un contenedor no es una VM ligera

No arranca un sistema operativo: aísla un proceso con su entorno sobre el kernel que ya corre en el host.



0

sistemas operativos extra por contenedor — comparte el kernel del host



GB → MB

del peso de una VM al de una imagen de contenedor



Segundos

de arranque — el proceso ya está compilado y listo

El contenedor de carga marítimo

Antes de 1956 cada carga era distinta. El envase estandarizado hizo universal al transporte — y al software le pasó lo mismo.



Antes

Cajas, sacos y barriles: **cada carga exigía su propio manejo** en cada puerto.

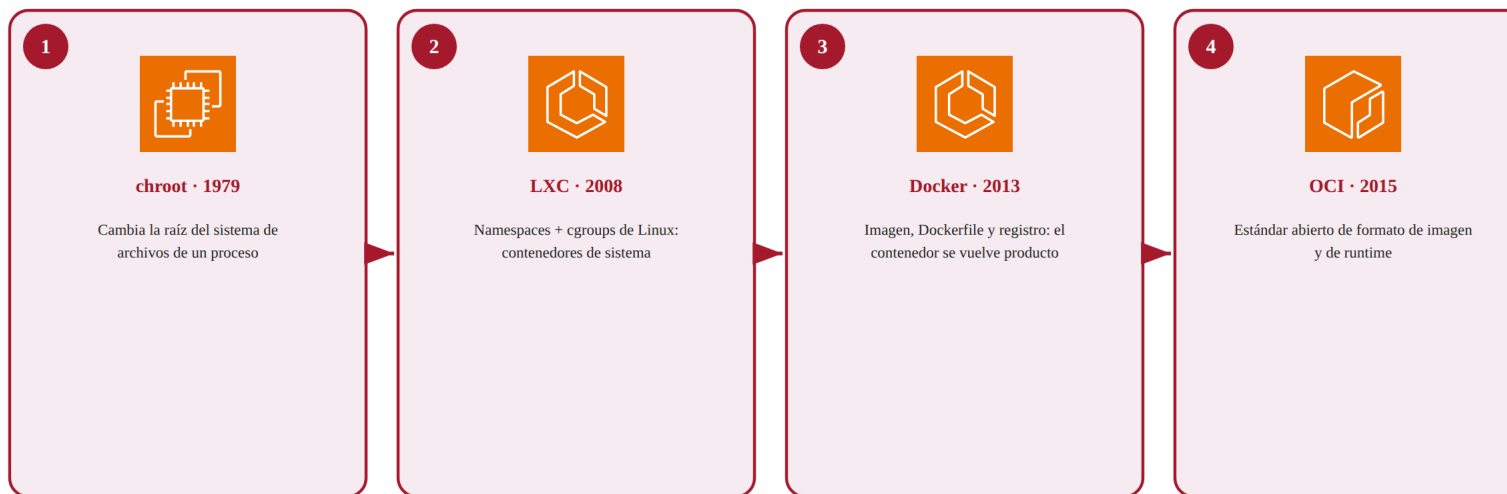
El estándar

El contenedor ISO tiene **el mismo formato en cualquier puerto** del mundo.

La grúa es el runtime

Mueve el contenedor **sin abrirlo** ni saber qué lleva adentro.

La idea no es nueva; el estándar abierto sí lo es — y eso es lo que hoy hace el tema genérico.



OCI: por qué el tema es genérico

El formato de imagen y la interfaz de runtime son un estándar abierto — la marca del registro o del orquestador es otra capa.



Formato de imagen

Misma especificación en Docker Hub, ECR, ACR o Artifact Registry



Runtime

containerd, CRI-O u otro que implemente la especificación



Herramienta

Cualquier builder compatible construye y ejecuta la imagen

Lo que aprendes hoy vale en cualquier proveedor: el concepto lo fija el estándar, no la marca.

El motor que ejecuta contenedores

Docker es la herramienta que usas; debajo hay un runtime estándar que ejecuta los contenedores.

```
node1 — Play with Docker

$ docker info | grep -i runtime
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
```

Docker

Trae el CLI que ya conoces y usa **containerd** como runtime por debajo.

En un clúster

Kubernetes suele usar **CRI-O** o containerd directamente contra el kubelet.

Sin daemon

Podman ejecuta contenedores sin proceso central — mismo formato de imagen.

La unidad de despliegue de casi cualquier plataforma moderna es la imagen de contenedor.



Escala real

Plataformas de streaming y transporte despliegan **miles de contenedores por día**.

Unidad de despliegue

Cada microservicio (Semana 7) va en **su propia imagen versionada**.

En el centro de datos

El orquestador **mueve contenedores entre máquinas** sin tocar la imagen.

El contenedor es la forma práctica de empaquetar y desplegar cada microservicio por separado.



La continuidad: la semana pasada el **qué** (un servicio por dominio); esta semana el **cómo** (una imagen por servicio).

Por qué existen los contenedores

Aislamiento de proceso, peso mínimo y un estándar abierto: eso es todo lo que sostiene el ecosistema.



Namespaces + cgroups

aislamiento real a nivel de proceso, sin SO extra
por app



Menos peso

MB y segundos frente a GB y minutos de la máquina
virtual



Estándar OCI

el mismo formato de imagen en cualquier runtime y
registro

Imagen vs. contenedor: el molde y el pastel

La imagen es el artefacto inmutable que se guarda y se comparte; el contenedor es una instancia en ejecución que se descarta.



Imagen

- Artefacto inmutable: capas de solo lectura
- Se guarda y se comparte en un registro
- Se identifica por tag y por digest



Contenedor

- Instancia en ejecución de una imagen
- Agrega una capa escribible encima
- Muere y nace; la imagen permanece

Cada instrucción del Dockerfile crea una capa; el contenedor solo escribe en la capa superior, sin tocar las demás.



copy-on-write

Capa escribible (contenedor)

Los cambios viven aquí — se pierden al borrar el contenedor

Capa de tu código (COPY)

Tu index.html — solo cambia cuando tú lo cambias

Capa de configuración (RUN)

Comandos congelados en el build



Imagen base (FROM)

nginx:alpine — compartida por todas las imágenes hijas

Dos archivos bastan para una app web estática: su contenido y el molde que la empaqueta.

```
node1 — mi-app/ — Play with Docker

$ cat Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html

$ cat index.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Mi primera app en contenedor</title>
  <style>body{font-family:sans-serif;display:flex;justify-content:center;align-items:center;height:100vh;background:#A6192E;color:#fff;margin:0}h1{font-size:3rem}</style>
</head>
<body>
  ...
```

FROM

La **imagen base** de partida — siempre una versión concreta.

COPY

Agrega archivos del contexto como **una capa nueva**.

El resultado

Una imagen identificada por `usuario/mi-app:1.0`.

Cinco instrucciones cubren la mayoría de las imágenes reales; el resto son variaciones de estas.

```
node1 — mi-app/ — Play with Docker

$ cat Dockerfile
FROM alpine:3.21
RUN mkdir -p /app && echo "listo" > /app/motd
COPY index.html /app/index.html
EXPOSE 8080
CMD ["cat", "/app/motd"]

$ docker build -t student01/mi-app:1.0 .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 171B done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/alpine:3.21
#2 DONE 2.9s
#3 [internal] load .dockerignore
#3 DONE 0.0s
#4 [internal] load build context
#4 DONE 0.0s
#5 [1/3] FROM docker.io/library/alpine:3.21@sha256:ce64758a109eb420d874a118f87920e625e12d3634e03b4a5573fd9f6e5d3507
#5 resolve docker.io/library/alpine:3.21@sha256:ce64758a109eb420d874a118f87920e625e12d3634e03b4a5573fd9f6e5d3507 0.0s done
#5 sha256:16333ee0c00fc65e025a2a4f839703ad37a74728832977fbdff080984de1b8e5a 0B / 3.63MB 0.2s
#5 sha256:16333ee0c00fc65e025a2a4f839703ad37a74728832977fbdff080984de1b8e5a 1.05MB / 3.63MB 0.5s
#5 sha256:16333ee0c00fc65e025a2a4f839703ad37a74728832977fbdff080984de1b8e5a 2.10MB / 3.63MB 0.6s
#5 sha256:16333ee0c00fc65e025a2a4f839703ad37a74728832977fbdff080984de1b8e5a 3.63MB / 3.63MB 0.6s done
#5 extracting sha256:16333ee0c00fc65e025a2a4f839703ad37a74728832977fbdff080984de1b8e5a 0.1s done
#5 DONE 0.8s
```

FROM

La imagen base de partida — **con tag explícito**, nunca latest.

RUN · COPY

Comandos congelados en capas y archivos agregados al build.

EXPOSE · CMD

El puerto que escucha y el proceso que arranca al correr.

El contexto de construcción

Todo lo que hay en la carpeta se envía al daemon de Docker — el `.dockerignore` define qué viaja y qué no.

```
node1 — mi-app/ — Play with Docker

$ ls -la peso.dat && cat .dockerignore
-rw-rw-r-- 1 cuc cuc 8388608 sep 26 06:00 peso.dat
peso.dat

$ docker build -t student01/mi-app:1.0 . # SIN .dockerignore
#1 [internal] load build definition from Dockerfile
#2 [internal] load metadata for docker.io/library/alpine:3.21
#3 [internal] load .dockerignore
#3 transferring context: 2B done
#5 [internal] load build context
#5 transferring context: 8.39MB 0.1s done

$ echo 'peso.dat' > .dockerignore && docker build --no-cache -t student01/mi-app:1.0 .
#3 transferring context: 49B done
#4 transferring context: 107B done
```

El contexto

La carpeta actual viaja al daemon — con BuildKit, solo lo que el Docker referencia.

.dockerignore

Excluye lo que no debe viajar: el ejemplo real de **8 MB → 107 B**.

Menos contexto

Build más rápido y sin filtrar basura hacia la imagen.

La caché de capas acelera todo

Si una instrucción no cambió, su capa se reutiliza — el orden del Dockerfile decide cuánto se reconstruye.

```
node1 — mi-app/ — Play with Docker

$ docker build -t student01/mi-app:1.0 .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 104B done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 0.0s
#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s
#4 [internal] load build context
#4 transferring context: 32B done
#4 DONE 0.0s
#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:1ed1b0e1d7652937d6cbdaf4018c7b6fc009a7dd6c3047351e2eddda745de43f
#5 resolve docker.io/library/nginx:alpine@sha256:1ed1b0e1d7652937d6cbdaf4018c7b6fc009a7dd6c3047351e2eddda745de43f 0.0s
done
```

CACHED

La capa sin cambios **se reutiliza** en vez de reconstruirse.

Ordena el Dockerfile

De lo más estable a lo más cambiante: base → dependencias → código.

El primer build

Descarga la base; los siguientes son **cuestión de segundos**.

Cada instrucción del Dockerfile es una etapa del build; el resultado se congela en una imagen con nombre y tag.

```
node1 — mi-app/ — Play with Docker

$ docker build -t student01/mi-app:1.0 .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 104B done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 0.0s
#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s
#4 [internal] load build context
#4 transferring context: 476B done
#4 DONE 0.0s
#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:1ed1b0e1d7652937d6cbdaf4018c7b6fc009a7dd6c3047351e2eddda745de43f
#5 resolve docker.io/library/nginx:alpine@sha256:1ed1b0e1d7652937d6cbdaf4018c7b6fc009a7dd6c3047351e2eddda745de43f 0.0s
done
#5 DONE 0.1s
#6 [2/2] COPY index.html /usr/share/nginx/html/index.html
#6 DONE 0.0s
#7 exporting to image
#7 exporting layers 0.0s done

$ docker history student01/mi-app:1.0
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
e293f1e44ebe   1 second ago    COPY index.html /usr/share/nginx/html/index.
<missing>      3 days ago     RUN /bin/sh -c set -x    && apkArch="$(cat _
<missing>      3 days ago     ENV ACME_VERSION=0.4.1   0B        buildkit.dockerfile.v0
<missing>      3 days ago     ENV NJS_RELEASE=1        0B        buildkit.dockerfile.v0
<missing>      3 days ago     ENV NJS_VERSION=1.0.1    0B        buildkit.dockerfile.v0
```

Las etapas

El build ejecuta el Dockerfile de arriba abajo y va publicando cada capa

naming to ...

La imagen final recibe **tu usuario, tu nombre y tu tag**.

docker history: la pila de tu imagen

La imagen final es exactamente la suma de sus capas — tu código es la más pequeña de todas.

```
node1 — mi-app/ — Play with Docker

$ docker history student01/mi-app:1.0
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
e293f1e44ebe   1 second ago    COPY index.html /usr/share/nginx/html/index... 24.6kB    buildkit.dockerfile.v0
<missing>      3 days ago      RUN /bin/sh -c set -x  && apkArch="$(cat ... 52.4MB    buildkit.dockerfile.v0
<missing>      3 days ago      ENV ACME_VERSION=0.4.1              0B        buildkit.dockerfile.v0
<missing>      3 days ago      ENV NJS_RELEASE=1                    0B        buildkit.dockerfile.v0
<missing>      3 days ago      ENV NJS_VERSION=1.0.1                0B        buildkit.dockerfile.v0
<missing>      3 days ago      CMD ["nginx" "-g" "daemon off;"]    0B        buildkit.dockerfile.v0
<missing>      3 days ago      STOPSIGNAL SIGQUIT                  0B        buildkit.dockerfile.v0
```

Tu capa arriba

El COPY de tu app pesa **kilobytes** sobre una base de decenas de MB.

La base debajo

Las capas de nginx:alpine — **se comparten** entre imágenes hijas.

Imágenes pequeñas: elige bien la base

Una base mínima reduce superficie de ataque, tiempo de pull y costo de almacenamiento en el registro.

```
node1 — Play with Docker

$ docker images --format 'table {{.Repository}}:{{.Tag}}\t{{.Size}}'
REPOSITORY:TAG      SIZE
nginx:alpine        94.4MB
alpine:3.21         12.2MB
ubuntu:24.04        119MB
```

Base mínima (alpine)

12 MB: arranque y pull rápidos, superficie mínima.

Base completa (ubuntu)

119 MB: solo si hay herramientas que realmente necesites.

La regla

Cada MB de base es superficie de ataque y tiempo de pull en cada despliegue.

La etapa de build pesa lo que tenga que pesar; la etapa de ejecución se queda solo con el resultado.

```
node1 — mi-app/ — Play with Docker

$ cat Dockerfile
FROM ubuntu:24.04 AS build
RUN dd if=/dev/zero of=/tmp/pesado.tmp bs=1M count=30 && echo "compilado" > /salida.txt
FROM alpine:3.21
COPY --from=build /salida.txt /salida.txt

$ docker build -t student01/mi-app:1.0 .
#5 [build 1/2] FROM docker.io/library/ubuntu:24.04@sha256:008173c23f95b170204355c12626cb5a965d779a7e1283b09e9cf
#6 [build 2/2] RUN dd if=/dev/zero of=/tmp/pesado.tmp bs=1M count=30 && echo "compilado" > /salida.txt
#7 [stage-1 1/2] FROM
docker.io/library/alpine:3.21@sha256:ce64758a109eb420d874a118f87920e625e12d3634e03b4a5573fd9f6e5d3507
#6 [build 2/2] RUN dd if=/dev/zero of=/tmp/pesado.tmp bs=1M count=30 && echo "compilado" > /salida.txt
#8 [stage-1 2/2] COPY --from=build /salida.txt /salida.txt
#9 exporting layers 0.0s done
#9 naming to docker.io/library/demo-multistage:1.0 done
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
demo-multistage:1.0  3d36de06ec17      12.1MB      3.63MB

$ docker images student01/mi-app:1.0
demo-multistage:1.0  3d36de06ec17  12.1MB  (la etapa build: 30MB+ no viajan)
```

Etapa de build

Puede pesar lo que quiera: compiladores, caches, temporales (30 MB en el ejemplo).

COPY --from

Solo el **resultado** pasa a la etapa final.

Resultado: 12 MB

El temporario de la etapa build **nunca viaja** en la imagen final.

No corras como root

Por defecto el proceso dentro del contenedor es root: si alguien escapa del aislamiento, sale como root.

```
$ cat Dockerfile
```

```
FROM alpine:3.21
RUN adduser -D appuser
USER appuser
CMD ["id"]
```

```
$ docker build -t student01/mi-app:1.0 . && docker run --rm student01/mi-app:1.0
```

```
#1 DONE 0.0s
#2 DONE 0.0s
#3 DONE 0.0s
#5 DONE 0.3s
#6 naming to docker.io/library/demo-noroot:1.0 done
#6 unpacking to docker.io/library/demo-noroot:1.0 0.0s done
#6 DONE 0.2s
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
```

USER appuser

El proceso corre con un usuario sin privilegios dentro del contenedor.

id lo demuestra

uid=1000(appuser) — sin privilegios.

Menos impacto

Si algo escapa del aislamiento, no sale como root.

Tags: latest se mueve, las versiones no

El tag es el nombre legible; el digest sha256 es la identidad real del contenido — inmutable.

```
node1 — Play with Docker

$ docker images --digests student01/mi-app
REPOSITORY      TAG      DIGEST                                     IMAGE ID      CREATED
SIZE
student01/mi-app 1.0      sha256:05c284709f1863f662b97495888fdad4a0618b60b957fdff821df6b91d5a77fe  e293f1e44ebe  1 minute
ago    93.6MB
student01/mi-app 2.0      sha256:af97455614a164b924daf0201878b1bcd2b2aea225c5c2adb64ecf5094b0215e  af97455614a1  just now
93.6MB
```

Repositorio

El nombre: usuario/mi-app.

Tag

1.0, 2.0... etiqueta legible — se puede mover (pero **no debes** en producción).

Digest

sha256:... — la huella del contenido: **eso** es lo que se despliega.

De la imagen al contenedor: build → run

Construir y correr es un ciclo corto: la imagen que acabas de construir arranca como cualquier otra.

```
node1 — Play with Docker

$ docker rm -f web
web

$ docker run -d -p 8080:80 student01/mi-app:1.0
f60ec1602a888ee424089f1268269ea21437229b479652942a86cd798b1b0ff1

$ curl -s localhost:8080
<!DOCTYPE html>
<html>
<head>
<body>
  <h1>Hola, soy Nombre Apellido</h1>
```

docker rm -f web

Fuera el contenedor genérico — **ahora corre tu imagen.**

Puerto 8080

Tu página con tu nombre, servida por nginx dentro del contenedor.

Dentro del contenedor: otro sistema de archivos

docker exec abre un proceso dentro del contenedor: lo que veas ahí no es tu instancia, es el contenedor.

```
node1 — Play with Docker

$ docker exec -it web sh
/ # cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.2
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
/ # exit
```

Alpine Linux

El contenedor trae **el sistema de archivos de su imagen**, no el de tu máquina.

El aislamiento

Procesos, red y archivos: lo que defines en la imagen y en el run.

Imagen inmutable por capas, molde reproducible en un Dockerfile, versión identificada por digest.



Capas

cada instrucción del Dockerfile es una capa
reutilizable en caché



Contexto

solo lo que el .dockerignore deja pasar llega a la
imagen



Multi-stage

compilar pesado, correr ligero; la imagen final solo
lleva el resultado

¿Qué es un registro de imágenes?

El registro es el almacén del que todas las máquinas descargan la misma imagen — el corazón del flujo de despliegue.



El almacén de imágenes

Todos bajan **la misma imagen** desde un solo lugar autorizado.

push → pull

Se publica una vez y se descarga en cualquier host.

Repositorios y tags

Organización, versionado y permisos por imagen.

Un mismo concepto, varios alojamientos

Cada nube ofrece su registro gestionado; el modelo mental es común: repositorio + tag + digest + autenticación.



Amazon ECR

Registro gestionado en AWS: escaneo, inmutabilidad de tags, permisos IAM



Azure Container Registry

Registro gestionado en Azure, acoplado a sus servicios de cómputo



Google Artifact Registry

Registro gestionado en Google Cloud para imágenes y artefactos

También existen: Docker Hub (el público que usamos hoy), GitHub GHCR y cualquier registro compatible OCI. **Mismo concepto — sin tabla de equivalencias: el estándar es el que manda.**

Tres nombres para la misma imagen: cómo la llamas, cómo la etiquetas y cómo se demuestra lo que contiene.

```
node1 — Play with Docker

$ docker images --digests student01/mi-app
REPOSITORY          TAG      DIGEST                               IMAGE ID      CREATED
SIZE
student01/mi-app    1.0      sha256:05c284709f1863f662b97495888fdad4a0618b60b957fdff821df6b91d5a77fe  e293f1e44ebe  1 minute
ago      93.6MB
student01/mi-app    2.0      sha256:af97455614a164b924daf0201878b1bcd2b2aea225c5c2adb64ecf5094b0215e  af97455614a1  just now
93.6MB
```

Repositorio

El nombre completo: `usuario/mi-app` — el dueño y la app.

Tag

Etiqueta legible que **se puede mover** — no es una identidad confiable por sí sola.

Digest

Hash del contenido: **immutable**. Lo que se despliega es el digest, no el tag.

Quién puede subir y quién puede bajar

Un registro sin permisos es un FTP público con capas dentro: el control de acceso es parte del diseño.

```
node1 — Play with Docker

$ docker push cuc-demo/mi-app:1.0    # sin permiso de escritura
The push refers to repository [docker.io/cuc-demo-noexiste/mi-app]
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
Error response from daemon: pull access denied for cuc-demo-noexiste/privado, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
'docker login'

$ docker pull cuc-demo/privado:1.0    # repo privado sin login
Error response from daemon: pull access denied for cuc-demo-noexiste/privado, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
'docker login'
```

push denegado

Sin permiso de escritura en el repositorio, el registro rechaza la publicación.

pull denegado

Un repositorio privado exige docker login antes de bajar.

La política

Identidad y permisos por repositorio — parte del diseño, no un adorno.

docker push sube las capas que faltan y devuelve el digest — la prueba de que el registro ya la tiene.

```
$ docker login
```

```
Login with your Docker ID to push and pull images from Docker Hub.  
If you don't have a Docker ID, head over to https://hub.docker.com to create one.  
Username: student01  
Password:  
Login Succeeded
```

```
$ docker push student01/mi-app:1.0
```

```
The push refers to repository [student01/mi-app]  
e76228b47809: Pushed  
6c53d0b2a666: Pushed  
745dfb2690dd: Pushed  
64c8194480fe: Pushed  
9a9a644fdd6a: Pushed  
ed37d4381da6: Pushed  
e2de96513ba9: Pushed  
d9aae54b5831: Pushed  
8bde0dc88c03: Pushed  
e72112c14215: Pushed  
1.0: digest: sha256:05c284709f1863f662b97495888fdad4a0618b60b957fdff821df6b91d5a77fe size: 856
```

Capa por capa

Las capas que el registro ya tiene se saltan: **solo sube lo nuevo**.

El digest

La línea final digest: sha256:... es la **huella criptográfica** de tu imagen.

Pull: la imagen vuelve a un host vacío

En una máquina que nunca construyó nada, docker pull descarga la imagen y arranca idéntica a como la dejaste.

```
node2 (instancia nueva) — Play with Docker

# Instancia NUEVA y vacía en Play with Docker:
$ docker pull student01/mi-app:2.0
2.0: Pulling from mi-app
495c9e424243: Pulling fs layer
e7761770e95d: Download complete
495c9e424243: Already exists
495c9e424243: Pull complete
Digest: sha256:af97455614a164b924daf0201878b1bcd2b2aea225c5c2adb64ecf5094b0215e
Status: Downloaded newer image for student01/mi-app:2.0
student01/mi-app:2.0

$ docker run -d -p 8080:80 student01/mi-app:2.0
0269935b0b70cd13a3191212a63314456d23a1ba4c19b89432d2a6d575d89048

$ curl -s localhost:8080 | grep h1
<h1>Hola, soy Nombre Apellido - v2.0</h1>
```

Sin build

El host nuevo **no compila nada**: descarga capas del registro.

Mismo digest

Lo que baja es **bit a bit** lo que subió — el digest lo garantiza.

Dónde publiques define quién ve tu imagen y cómo se audita su uso.



Registro público

- Imágenes de base y de terceros
- Cualquiera puede hacer pull
- Si tu app es privada, no la subas aquí



Registro privado

- Imágenes de tu organización
- Pull y push con autenticación
- Auditoría de quién publica qué, y cuándo

Las capas heredadas traen sus propios CVEs: el registro puede analizarlas antes de que nadie las despliegue.



Escaneo del registro

Analiza cada capa por vulnerabilidades conocidas (CVE)



Hallazgos accionables

Severidad, capa afectada y paquete responsable



Política de puerta

No desplegar imágenes con vulnerabilidades severas abiertas

Firmar una imagen es atarla criptográficamente a quien la construyó: el runtime puede exigir esa firma.

**Firma**

El pipeline firma la imagen con una clave que solo él tiene

**Verificación**

El runtime rechaza las imágenes sin firma válida
— o las firma el atacante

**Cadena de confianza**

Del repositorio de código al registro, sin pasos manuales

Las imágenes se acumulan: tags viejos, capas huérfanas y caché de build ocupan disco en cada host.

```
node1 — Play with Docker

$ docker system df
TYPE                TOTAL        ACTIVE        SIZE           RECLAIMABLE
Images              14           7            3.288GB       1.28GB (39%)
Containers          8           7            237.6kB       4.096kB (1%)
Local Volumes       3           3             76.18MB       0B (0%)
Build Cache        13           0            26.41MB       57.85kB
```

docker system df

Muestra qué ocupan imágenes, contenedores, volúmenes y caché.

docker system prune

Borra lo que **no usa ningún contenedor** — siempre con la lista a la vista antes.

En la práctica del curso

El lab a demanda de ASUR borra todo con `lab down`: solo sobrevive el CSV de evidencia.

Versión 2: las capas base ya existen

Al editar solo tu página, solo esa capa cambia — el push reutiliza todo lo demás.

```
node1 — Play with Docker

$ docker build -t student01/mi-app:2.0 .
#7 exporting attestation manifest sha256:3985d7037529a9fef1237f22181da834d778ea2b48037784d73998cbb8047982 0.0s done
#7 exporting manifest list sha256:af97455614a164b924daf0201878b1bcd2b2aea225c5c2adb64ecf5094b0215e done
#7 naming to docker.io/student01/mi-app:2.0 done
#7 unpacking to docker.io/student01/mi-app:2.0 0.0s done
#7 DONE 0.1s

$ docker push student01/mi-app:2.0
The push refers to repository [student01/mi-app]
495c9e424243: Pushed
e76228b47809: Layer already exists
9a9a644fdd6a: Layer already exists
d9aae54b5831: Layer already exists
64c8194480fe: Layer already exists
6c53d0b2a666: Layer already exists
2.0: digest: sha256:af97455614a164b924daf0201878b1bcd2b2aea225c5c2adb64ecf5094b0215e size: 856

# En hub.docker.com → repositorio student01/mi-app, pestaña Tags:
{"name":"mi-app","tags":["2.0","1.0"]}
```

Layer already exists

Las capas idénticas **ya están** en el registro: el push es casi instantáneo

Dos tags, dos digests

La 1.0 y la 2.0 coexisten — **un tag nunca se sobrescribe**.

Si la versión nueva falla, el despliegue anterior es un simple cambio de digest — nada se reconstruye.

```
node1 — Play with Docker

# la versión 3.21 tiene un fallo — se vuelve a la anterior:
$ docker run --rm alpine:3.21 cat /etc/alpine-release
3.21.8
$ docker run --rm alpine:3.20 cat /etc/alpine-release
3.20.10

$ docker pull alpine:3.20    # rollback = volver al tag anterior
3.20: Pulling from library/alpine
Status: Image is up to date for alpine:3.20
docker.io/library/alpine:3.20
```

Dos tags

3.21 y 3.20 coexisten: **nada se sobrescribe.**

El fallo

La versión nueva falla en producción.

Rollback

Volver al tag anterior es un `pull` y un `run` — sin reconstruir nada.

El mismo concepto gestionado: ECR como ejemplo

En la nube el registro deja de ser un servidor que administras: sigue siendo repositorio + tag + digest, con más controles.



Amazon ECR

Registro gestionado: replicación, escaneo, inmutabilidad de tags



Permisos por repositorio

Quién sube y quién baja, resuelto con identidad del proveedor



Integración de cómputo

Los servicios de cómputo hacen pull directo del registro

Lo genérico: el concepto de registro es el estándar OCI — ECR es el ejemplo de este curso, no el único camino.

De dónde salen las imágenes base

Tu imagen empieza en otra imagen: confiar en la base es la primera decisión de seguridad del build.



Imágenes oficiales

Mantenedores conocidos, Dockerfile público,
parches periódicos



Verifica antes de confiar

Revisa CVEs abiertos y la actividad reciente del
mantenedor



Tu propia base

Congela una versión conocida y parchéala en tu
pipeline, no a mano

Publicar una imagen es tan importante como construirla: sin registro no hay portabilidad ni rollback.



Repositorio + tag + digest

los tres nombres de una imagen en el registro



Escanea y firma

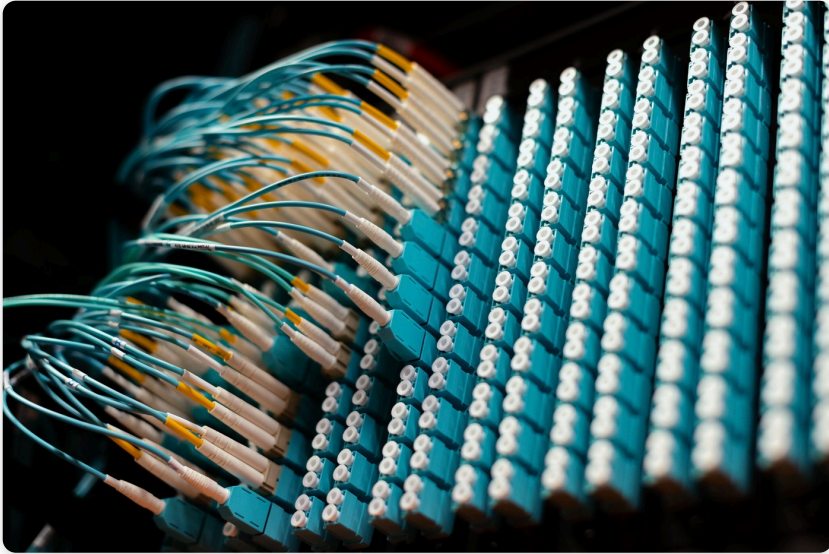
antes de que el orquestador jale tu imagen



Limpia lo que no usas

el disco de cada host se llena de capas olvidadas

El contenedor es efímero por diseño: la red lo conecta y el volumen conserva lo que debe sobrevivirle.



Redes

Puertos publicados y **DNS interno por nombre** de contenedor.

Volúmenes

Los datos viven fuera de la capa escribible y **sobreviven al contenedor**.

El contrato

Lo efímero se descarta; lo persistente se monta desde fuera.

La red de Docker: se encuentran por nombre

Dos contenedores en la misma red se resuelven por nombre — sin IPs manuales ni archivos de configuración.

```
node1 — Play with Docker

$ docker network create red
e7854e7ce39c7638a48c02245eae0c9ddcf408a7016f134983c35a912a46c21

$ docker run -d --name web2 --network red nginx:alpine
b6fb36f5cb3519f6c151ec46e457f5d48a60c322329aaf0de4121035caeaec4d

$ docker run --rm --network red busybox ping -c 3 web2
37bb94b0940b: Pull complete
PING web2 (172.22.0.2): 56 data bytes
64 bytes from 172.22.0.2: seq=0 ttl=64 time=0.081 ms
64 bytes from 172.22.0.2: seq=1 ttl=64 time=0.062 ms
64 bytes from 172.22.0.2: seq=2 ttl=64 time=0.068 ms

--- web2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.062/0.070/0.081 ms
```

docker network create

Crea una red aislada donde los contenedores se ven entre sí.

DNS por nombre

ping web2 resuelve al contenedor — **sin IP escrita a mano**.

compose.yaml describe el conjunto: servicios, imágenes, puertos y redes — reproducible con un comando.

```
node1 — mi-app/ — Play with Docker

$ cat compose.yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
  redis:
    image: redis:alpine

$ docker compose up -d
[+] Running 3/3
 ✓ Network mi-app_default    Created
 ✓ Container mi-app-web-1    Started
 ✓ Container mi-app-redis-1  Started

$ docker compose ps
NAME                IMAGE              COMMAND                SERVICE  CREATED      STATUS      PORTS
compose-demo-redis-1  redis:alpine       "docker-entrypoint.s..." redis    1 second ago Up Less than a second 6379/tcp
compose-demo-web-1    nginx:alpine        "/docker-entrypoint..." web      1 second ago Up Less than a second 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

El archivo

Cada bloque `services:` es un contenedor del conjunto.

Un comando

`docker compose up -d` levanta todo; `down` lo borra todo.

Compose no es un orquestador: es la forma de tener el mismo conjunto de servicios en cada máquina del equipo.



Servicios

Cada bloque del compose.yaml es un contenedor y su configuración



Volúmenes y redes

Declarados junto a los servicios, en el mismo archivo

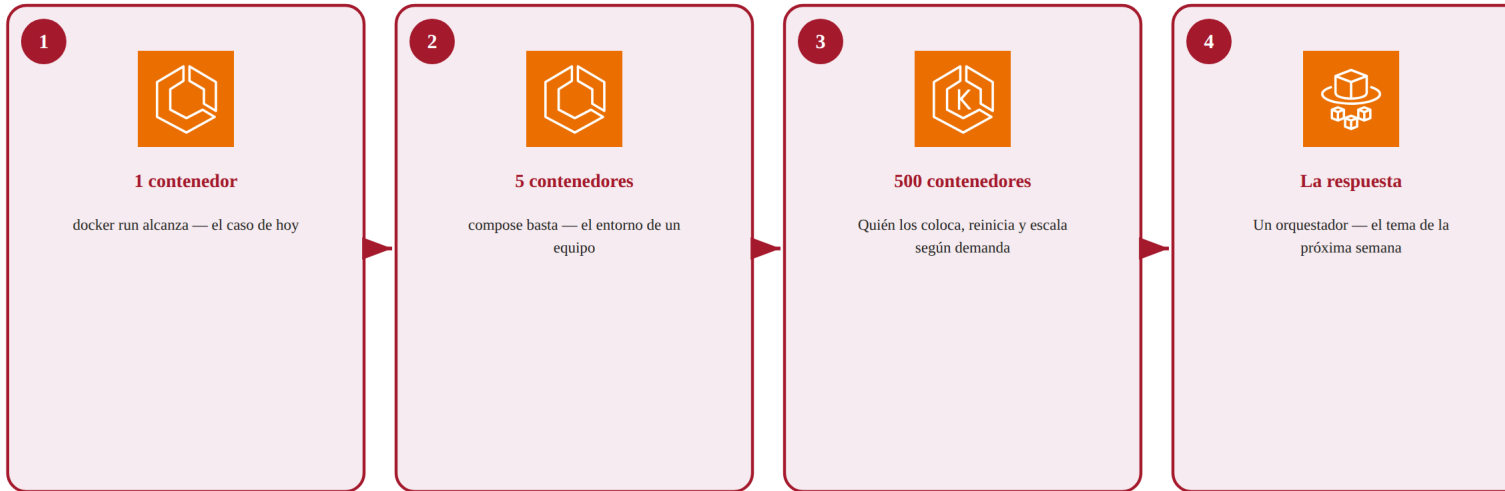


Ciclo corto

up, ps, down: el entorno completo se levanta y se descarta sin rastro

¿Por qué luego hace falta orquestación?

Compose basta para un equipo; cuando el sistema crece, hace falta quién coloque, reinicie y escale contenedores.



El orquestador es la plataforma que sostiene los contenedores en producción: programación, red, escalamiento y recuperación.



Kubernetes

Estándar abierto de orquestación: pods, deployments, services



Servicios gestionados

EKS, AKS, GKE, OKE: Kubernetes operado por el proveedor de nube



Cómputo sin servidores

El proveedor corre tus contenedores sin que administres nodos

Sin tablas de equivalencias: el concepto de orquestación es común a todos; la operación concreta de cada servicio, no.

La imagen es el artefacto portable entre nubes

Lo que se construye una vez corre donde haya un runtime compatible — la nube es intercambiable por diseño.



El envase universal

La misma imagen corre en tu portátil, en tu centro de datos y en **cualquier nube**.

Lo único que cambia

El runtime y el registro que la sirve — **no el artefacto**.

La consecuencia

Despliegue multicloud deja de ser una promesa: es copiar el digest.

Build once, run anywhere en la práctica

El pipeline construye y publica una sola vez; cada ambiente toma su copia del registro y la corre.



Build — una vez

El pipeline construye y versiona la imagen de tu app.

Push — al registro

Tag + digest: la **identidad** del artefacto.

Deploy — en cada ambiente

Cada uno hace pull y reemplaza el contenedor.

Run — mismo digest

Desarrollo, pruebas y producción corren **exactamente lo mismo**.

Cada eslabón — código, imagen, runtime — necesita su propio control: la confianza no se hereda sola.



Código

Control de versiones con revisión: la primera capa de confianza.

Imagen

Build automatizado, escaneo y firma — sin builds manuales a producción.

Runtime

Solo imágenes verificadas arrancan; la red limita a dónde pueden llamar.

La guía completa está en Moodle — vale 10 % de la nota como Actividad 1 de U2.



El recorrido

- Pasos 1–3: primer contenedor, puertos y aislamiento
- Pasos 4–8: Dockerfile, build y push de dos versiones
- Paso 9: portabilidad en instancia nueva
- Paso 10: reto multi-contenedor

Entregable: URL de tu imagen en Docker Hub con sus dos tags + captura de docker ps + captura de tu app en el navegador. **Plan B:** si Play with Docker dice «capacity», los mismos comandos en Killercodea.

Tres ideas que sostienen todo lo que viene en U2 — y que ya puedes probar con tu propia imagen.



Imagen inmutable

empaqueta la app con su entorno, una vez, para cualquier lugar



Registro

el artefacto vive fuera de los servidores y viaja con un pull



Portabilidad

build once, run anywhere — la base del despliegue multicloud

Próxima semana — Semana 9: orquestación de contenedores (conceptos de Kubernetes y servicios gestionados). Tu imagen de hoy es exactamente lo que el orquestador va a mo